

Credit Card Approval Prediction

(by Using Machine Learning)

Project Description:

Credit Card Approval Prediction Using Machine Learning is an AI-powered web application that automates the credit card approval process by predicting whether an applicant is eligible for a credit card based on their financial and personal information. The system uses Machine Learning algorithms such as Logistic Regression, Decision Tree, Random Forest, and XGBoost to analyse historical applicant data and generate accurate approval predictions. The best-performing model is deployed using Flask, enabling users to receive real-time approval or rejection results through a simple and user-friendly interface. By reducing manual effort and improving decision-making speed, the system helps financial institutions process applications more efficiently and consistently.

Problem Statement

Manual credit card approval processes are inconsistent, slow, and prone to subjective bias, which can result in creditworthy applicants being rejected or high-risk applicants being approved. There is a need for a data-driven, automated system that can accurately classify credit card applications as approved or rejected based on applicant attributes, thereby improving decision speed, consistency, and reliability for financial institutions.

Scenarios:

Scenario 1: Standard Credit Card Application

A customer applies for a credit card by entering their employment details, annual income, education level, housing status, and family information through the web application. The system preprocesses the input data and uses the trained Machine Learning model to predict whether the application should be **Approved** or **Rejected**, displaying the result instantly.

Scenario 2: High-Risk Applicant Evaluation

A bank officer evaluates an applicant with an unstable employment history and relatively low annual income. After submitting the applicant's details, the system analyses multiple financial and demographic factors using the trained model and predicts a **Rejected** status, helping the bank minimize potential financial risk and make informed lending decisions.

Scenario 3: Bulk Customer Screening

A financial institution processes numerous credit card applications daily. Bank employees enter each applicant's information into the system, which rapidly evaluates every application using the deployed Machine Learning model. The application provides immediate approval

predictions, significantly reducing manual verification time, improving operational efficiency, and maintaining consistent decision-making across all applications.

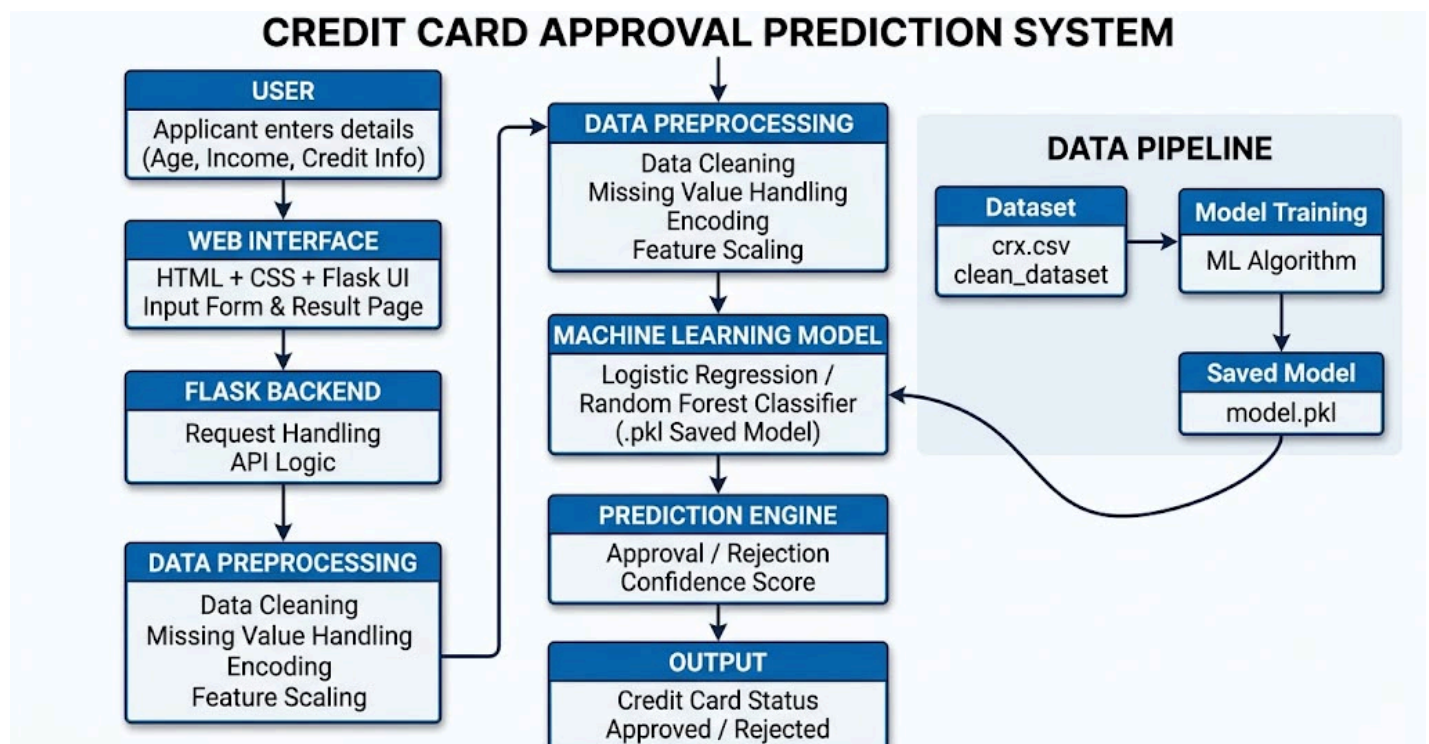
Objectives

- Automate the credit card approval prediction process using Machine Learning.
- Improve decision-making speed and reduce manual effort.
- Predict whether a credit card application is approved or rejected with good accuracy.
- Provide real-time predictions through a Flask-based web application.
- Enhance user experience with an easy-to-use and responsive interface.

Scope

The scope of this project is to develop a machine learning-based system that predicts whether a credit card application will be approved or rejected based on applicant details. The system helps financial institutions automate the initial screening process, reduce manual effort, and support faster decision-making. The project includes data preprocessing, model training, prediction, and a user-friendly web interface for entering applicant details and viewing results. Future enhancements may include real-time banking integration and advanced fraud detection features.

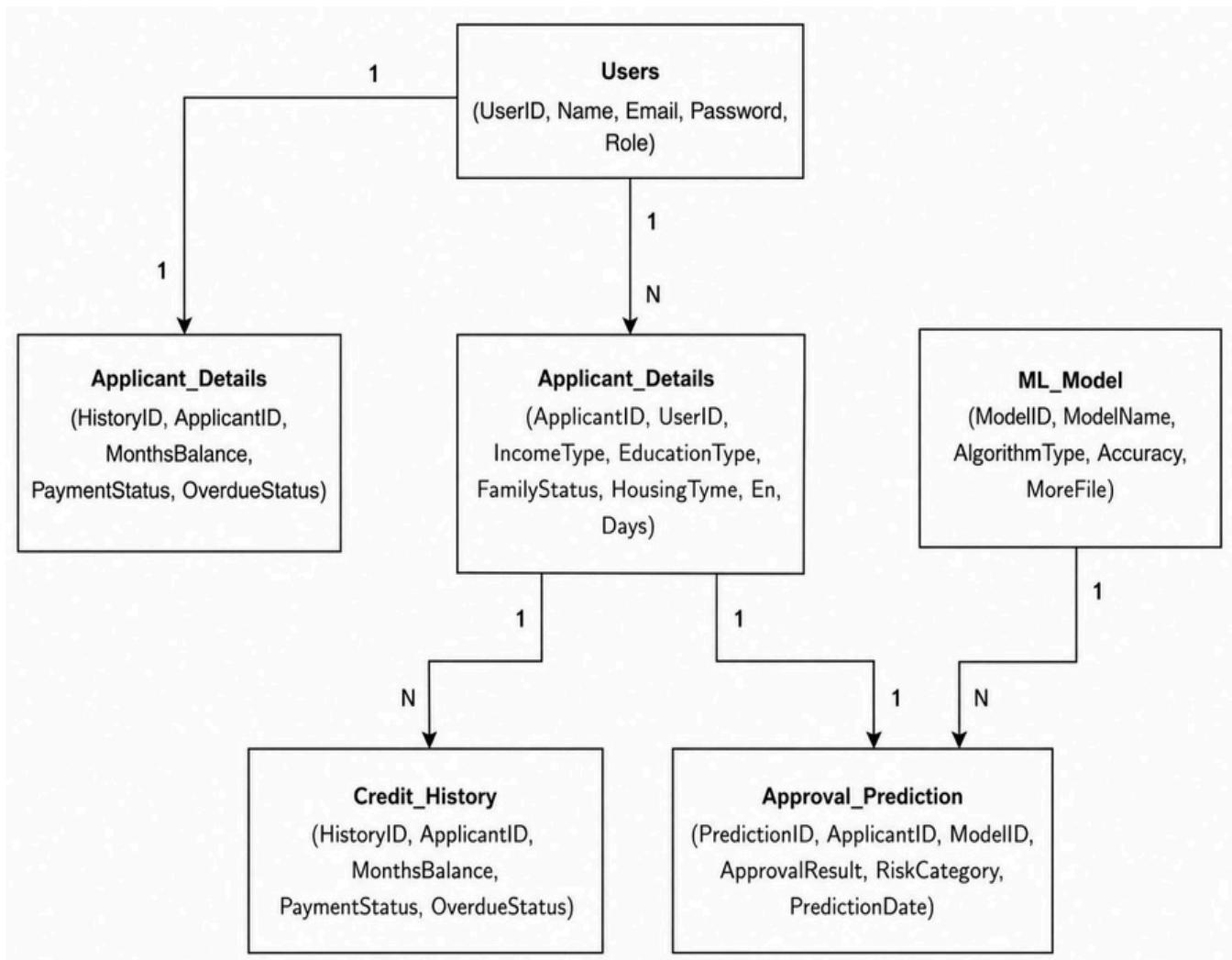
Technical Architecture: Architecture Flow



ER Diagram

Description:

The Entity Relationship (ER) diagram of the Credit Card Approval Prediction System represents the relationship between applicants, credit card applications, and machine learning prediction results. The Applicant entity stores the personal and financial details of customers, such as age, gender, income, occupation, and credit history. Each applicant can submit one or more Credit Card Applications, which contain application-related information and credit details. The submitted application data is processed by the machine learning model, and the generated output is stored in the ML Prediction entity, which includes the prediction result (Approved/Rejected), confidence score, model details, and prediction date. The relationship between these entities helps organize applicant information, application records, and prediction outcomes efficiently, providing a structured view of the credit card approval decision-making process.



Pre-requisites

- Python Programming Proficiency (Python 3.x)
- Machine Learning Fundamentals — scikit-learn, XGBoost
- Data Handling Libraries — Pandas, NumPy
- Data Visualization — Matplotlib / Seaborn (for EDA)
- Web Framework Knowledge — Flask, for deployment of the prediction interface
- Version Control with Git

Project Workflow:

Milestone 1: Data Collection and Understanding :

- Activity 1.1: Dataset Collection: Collected credit card application dataset containing applicant and financial details.
- Activity 1.2: Dataset Loading: Loaded the dataset into Python using Pandas for analysis.
- Activity 1.3: Data Exploration: Analyzed dataset structure, features, and basic statistics.
- Activity 1.4: Data Quality Check: Identified missing values, duplicates, and inconsistencies.
- Activity 1.5: Feature Understanding: Studied important attributes affecting credit card approval prediction.

Milestone 2: Data Preprocessing and Feature Engineering

- Activity 2.1: Data Cleaning
Handled missing values, duplicates, and inconsistent data.
- Activity 2.2: Data Transformation
Converted categorical features into numerical format.
- Activity 2.3: Feature Engineering
Selected and optimized important features for prediction.
- Activity 2.4: Dataset Preparation
Split the processed data into training and testing sets for model development.

Milestone 3: Model Building

- Activity 3.1: Model Selection
Selected suitable machine learning classification algorithms for credit card approval prediction.
- Activity 3.2: Model Training
Trained the model using the preprocessed dataset.
- Activity 3.3: Model Evaluation
Evaluated model performance using accuracy and other evaluation metrics.
- Activity 3.4: Model Saving
Saved the trained model as a .pkl file for prediction and deployment.

Milestone 4: Model Evaluation and Selection

- Activity 4.1: Evaluate each trained model on the test set using Accuracy, Precision, Recall, and F1-Score.
- Activity 4.2: Generate a Confusion Matrix for each model to analyse true/false positive and negative predictions.
- Activity 4.3: Compute the ROC-AUC score to assess the models' ability to discriminate between classes.
- Activity 4.4: Compare all models and select the best-performing algorithm for deployment. Based on the evaluation results (detailed in Section 5 — Results), the XGBoost Classifier achieved the highest accuracy of 89.86% and was selected as the final model for deployment.

Milestone 5: Deployment

- Activity 5.1: Save the best-performing trained model (XGBoost) using joblib for reuse in the web application.
- Activity 5.2: Build a Flask-based web application with an HTML/CSS front-end that collects applicant details through an application form.
- Activity 5.3: Integrate the trained model with the Flask backend (app.py) to generate real time approval predictions via a /predict API endpoint.
- Activity 5.4: Deploy the application on Render, making it publicly accessible as a live web service.
- Activity 5.5: Test the live application end-to-end to verify correct functioning, from form submission to prediction display.

The application has been successfully deployed and is live at the following URL:

<https://credit-card-approval-prediction-y76r.onrender.com/>

Full implementation details, source code, and screenshots of the deployed web application are provided in Section 6 (Web Application).

Methodology

Dataset Description

Attribute	Details
Dataset Name	Credit Card Approval Dataset (crx.csv / clean_dataset.csv)
Type	Structed
Target Variable	Credit Card Approval Status(accepted/rejected)
Feature Types	Categorical & Numerical attributes of attributes

Data Preprocessing Steps

- Data Loading and Inspection:

The credit card approval dataset was loaded using Pandas and analyzed using functions such as `head()`, `info()`, and `describe()` to understand the dataset structure, features, and statistical information.

- Missing Value Verification:

The dataset was checked for missing values and appropriate handling techniques were applied to ensure data quality.

- Duplicate Checking:

Duplicate records were identified and removed to improve data consistency and prevent biased predictions.

- Categorical Encoding:

Categorical features such as Gender, Education, Marital Status, and Credit History were converted into numerical format using encoding techniques.

- Feature Selection:

Important features influencing credit card approval decisions were selected for model training.

- Train-Test Split:

The dataset was divided into training and testing sets using `train_test_split` for model evaluation.

- Feature Scaling:

Numerical features were scaled using techniques such as `StandardScaler` to improve machine learning model performance.

Algorithms Used

- 🚦 Logistic Regression (Final deployed model)
- 🚦 Random Forest Classifier (Model comparison)
- 🚦 Decision Tree Classifier (Model comparison)

Evaluation Metrics

- Accuracy — Overall proportion of correctly classified applications.
- Precision — Proportion of predicted approvals that were actually correct.
- Recall — Proportion of actual approvals that were correctly identified.
- F1-Score — Harmonic mean of Precision and Recall, useful for imbalanced classes.
- Confusion Matrix — Detailed breakdown of true/false positives and negatives
- ROC-AUC — Measures the model's ability to distinguish between approved and rejected classes across thresholds

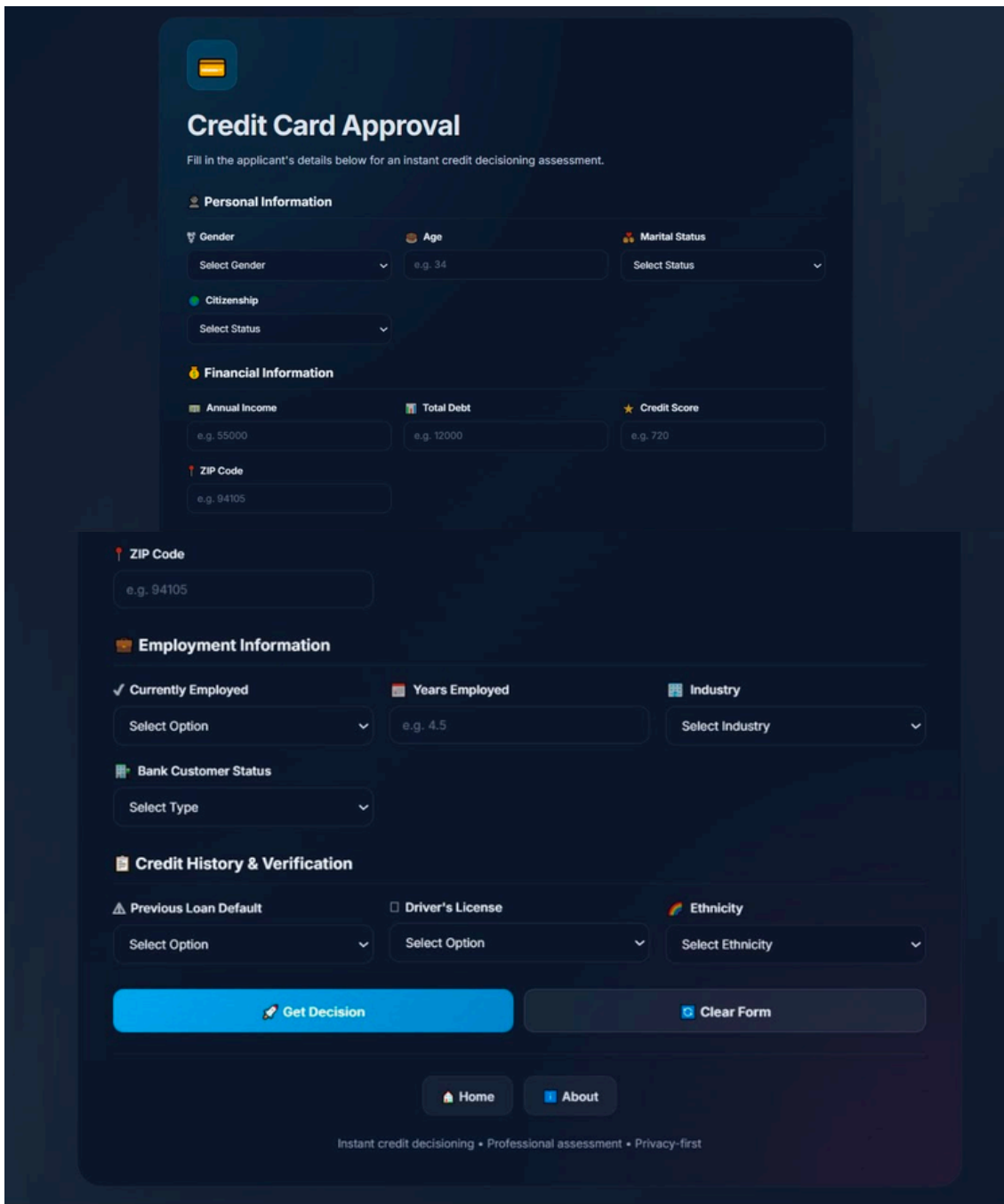
Web Application

The trained Logistic Regression model has been deployed as an interactive web application named "Credit Card Approval Prediction", built using Flask (backend) and HTML/CSS/JavaScript (frontend). The application allows users to enter applicant details through a web form and instantly predicts whether the credit card application will be Approved or Rejected, along with the prediction confidence score. The system also provides a brief explanation of the prediction based on the applicant's profile.

The application has been successfully deployed and is publicly accessible at:

<https://credit-card-approval-prediction-y76r.onrender.com/>

Application Form



The screenshot displays the "Credit Card Approval" web application form. The form is titled "Credit Card Approval" and includes a subtitle: "Fill in the applicant's details below for an instant credit decisioning assessment." The form is organized into several sections:

- Personal Information:** Includes fields for Gender (dropdown), Age (text input, e.g., 34), Marital Status (dropdown), and Citizenship (dropdown).
- Financial Information:** Includes fields for Annual Income (text input, e.g., 55000), Total Debt (text input, e.g., 12000), Credit Score (text input, e.g., 720), and ZIP Code (text input, e.g., 94105).
- Employment Information:** Includes fields for Currently Employed (checkbox), Years Employed (text input, e.g., 4.5), Industry (dropdown), and Bank Customer Status (dropdown).
- Credit History & Verification:** Includes fields for Previous Loan Default (checkbox), Driver's License (checkbox), and Ethnicity (dropdown).

At the bottom of the form, there are two buttons: "Get Decision" (blue) and "Clear Form" (grey). Below the buttons are links for "Home" and "About". The footer text reads: "Instant credit decisioning • Professional assessment • Privacy-first".

Prediction of Result



Approved

High confidence prediction.

 **Confidence Score**

Decision Confidence

100.0%

 **Decision Summary**

Your overall profile shows sufficient financial stability for this credit product.

▼ Your Strengths

- Strong employment profile: You have 4.5 years of employment experience.
- Your very good credit score shows solid creditworthiness.
- Adequate income level: Your annual income of \$55,000 is sufficient for credit management.

 **Recommended Next Steps**

- Consider reapplying after 6 months with improved financial metrics.

 **Application Summary**

AGE	ANNUAL INCOME	TOTAL DEBT	CREDIT SCORE	YEARS EMPLOYED
45	\$55000	\$12000	720	4.5
EMPLOYMENT				
Yes				

Apply Again

Home

About

Activate Windows
Go to Settings to activate Windows.

- **HTML**
- **CSS**
- **JavaScript**

The image shows the Android Studio IDE with a project named 'Smart bridge project'. The main editor window displays the XML code for a layout file named 'home.html'. The code defines a Card Approval screen with a title, a link text, and a button. The button has a gradient background, rounded corners, and a shadow. The code is as follows:

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
3     android:layoutWidth="1000px"
4     android:layoutHeight="1000px"
5     android:background="#FFFFFF"
6     android:padding="10px">
7     <TextView
8         android:id="@+id/title"
9         android:layoutWidth="300px"
10        android:layoutHeight="50px"
11        android:text="Card Approval"
12        android:textColor="#000000"
13        android:textSize="24px"
14        android:gravity="center"
15        android:padding="10px">
16    </TextView>
17    <TextView
18        android:id="@+id/link_text"
19        android:layoutWidth="300px"
20        android:layoutHeight="50px"
21        android:text="Link Text"
22        android:textColor="#000000"
23        android:textSize="16px"
24        android:gravity="center"
25        android:padding="10px">
26    </TextView>
27    <Button
28        android:id="@+id/button"
29        android:layoutWidth="300px"
30        android:layoutHeight="50px"
31        android:text="Button Text"
32        android:textColor="#FFFFFF"
33        android:textSize="16px"
34        android:background="#000000"
35        android:backgroundRadius="10px"
36        android:padding="10px">
37    </Button>
38 </RelativeLayout>
```

The Project Explorer on the left shows the project structure, including the 'home.html' file. The Run toolbar at the bottom shows the 'Run' button (a green play icon) and other standard Android Studio icons.

The screenshot shows a VS Code editor window titled "Smart bridge project". The editor is open to a file named "index.html" (though the content is CSS). The left sidebar shows a file explorer with a "requirements.txt" file. The main editor area displays the following CSS code:

```

1  @font-face {
2    font-family: 'Inter';
3    src: url('https://fonts.gstatic.com/s/inter/v12/Inter.woff2');
4    font-weight: normal;
5    font-style: normal;
6  }
7
8  * {
9    margin: 0;
10   padding: 0;
11 }
12
13 body {
14   min-height: 100vh;
15   background: linear-gradient(135deg, #f0f2f1 49%, #e0e2e1 49%, #e0e2e1 51%, #f0f2f1 51%);
16   display: flex;
17   justify-content: center;
18   align-items: center;
19   padding: 40px 20px;
20   color: #212121;
21 }
22
23 .page-wrapper {
24   width: 100%;
25   max-width: 1000px;
26 }
27
28 .container {
29   width: 100%;
30   background: #fff;
31   border: 1px solid #ccc;
32   border-radius: 12px;
33   padding: 40px;
34   box-shadow: 0 4px 8px #ccc;
35   position: relative;
36   overflow: hidden;
37 }
38
39 .container::before {
40   content: '';
41   position: absolute;
42   inset: 0;
43   background: radial-gradient(circle at top left, #fff 49%, #ccc 49%, #ccc 51%, #fff 51%),
44     radial-gradient(circle at bottom right, #fff 49%, #ccc 49%, #ccc 51%, #fff 51%);
45 }
46
47 .header {
48   position: relative;
49   padding: 20px;
50   margin-bottom: 20px;
51 }
52
53 .header::after {
54   width: 220px;

```

The image shows a VS Code editor window with a project named "AI-ML-and-GEN-AI-Track-Project". The Explorer sidebar on the left displays the file structure, including a "templates" folder containing "about.html", "home.html", and "index.html". The "result.html" file is selected and its content is shown in the main editor. The code is an HTML template using Tailwind CSS, featuring a header, a result banner, a content section with actions, and a grid for applicants. The interface is in dark mode.

```

1  <html lang="en">
2  <head>
3  <style>
4  @media(max-width:768px){
5      .result-banner{
6      }
7      .result-title{
8          font-size:32px;
9      }
10     .content-section, .actions{
11         padding:24px;
12     }
13     .applicant-grid{
14         grid-template-columns:repeat(auto-fit, minmax(110px, 1fr));
15     }
16 }
17 @media(max-width:480px){
18     body{
19         padding:16px;
20     }
21     .result-banner{
22         padding:32px 16px;
23     }
24     .status-badge{
25         width:80px;
26         height:80px;
27         font-size:40px;
28     }
29     .result-title{
30         font-size:26px;
31     }
32     .content-section, .actions{
33         padding:16px;
34     }
35     .actions{
36         flex-direction:column;
37     }

```

```
le Edit Selection View Go ... ← → Smart bridge project ● 3 🔍 🏠 📁 🗑️ -
```

```
# requirements.txt # clean_dataset.csv ⚡ Coding & Solution .pdf ➦ app.py ➦ index.html ➦ result.html 4 X
```

```
AI-ML-and-GEN-AI-Track-Project > S. Project Development Phase > templates > ➦ result.html > ➦ html > ➦ head > ➦ style > {} @media (max-width:480px) > ➦ .content-section > ➦ .actions
```

```
2   <html lang="en">
```

```
3   <head>
```

```
6   <style>
```

```
267 }>
```

```
268 .actions{
```

```
269     padding:24px 48px;
```

```
270     background:linear-gradient(to right, #f0e68c 15%, #d4edda 23%, #d1ecf1 42%, #d4edda 57%);
```

```
271     border-top:1px solid black;
```

```
272     display:flex;
```

```
273     gap:14px;
```

```
274     flex-wrap:wrap;
```

```
275     justify-content:center;
```

```
276 }
```

```
277 .btn{
```

```
278     display:inline-flex;
```

```
279     align-items:center;
```

```
280     justify-content:center;
```

```
281     gap:8px;
```

```
282     padding:14px 28px;
```

```
283     border:none;
```

```
284     border-radius:12px;
```

```
285     font-weight:700;
```

```
286     font-size:15px;
```

```
287     cursor:pointer;
```

```
288     transition:all 0.2s ease;
```

```
289     text-decoration:none;
```

```
290 }
```

```
291 .btn-primary{
```

```
292     background:#007bff;
```

```
293     color:white;
```

```
294 }
```

```
295 .btn-primary:hover{
```

```
296     background:#0056b3;
```

```
297     transform:translateY(-2px);
```

```
298 }
```

```
299 .btn-secondary{
```

Backend Implementation Flask(app.py)

```

Edit Selection View Go ... Smart bridge project
venv C:\Users\Ansh\Desktop\Python\Projects\SmartBridge\venv
app.py x introduction results x
# Importing Flask and Jinja2
import logging
import os
from flask import Flask, render_template, request

app = Flask(__name__)

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s %(levelname)s %(message)s'
)
logger = logging.getLogger(__name__)

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
MODEL_PATH = os.path.join(BASE_DIR, 'model.pkl')
ENCODER_PATH = os.path.join(BASE_DIR, 'encoder.pkl')

def load_artifacts():
    """Load the serialized model and encoders from local assets."""
    try:
        model = joblib.load(MODEL_PATH)
        encoders = joblib.load(ENCODER_PATH)
        logger.info('Loaded model from %s', MODEL_PATH)
        logger.info('Loaded encoders from %s', ENCODER_PATH)
        logger.info('Model classes: %s', list(getattr(model, 'classes_', [])))
        return model, encoders
    except Exception:
        logger.exception('Failed to load model or encoders')
        raise

model, encoders = load_artifacts()
feature_order = list(
    getattr(
        model,
        'feature_names_in_',
        [
            'Gender',
            'Age',
            'Married',
            'MarriedStatus',
            'Industry',
            'Ethnicity',
            'Transacted',
            'PriorDefault',
            'Delinquent',
            'CreditScore',
            'DriverLicense',
            'Vehicle'
        ]
    )
)

@app.route('/')
def index():
    """Home page"""
    return render_template('index.html')

@app.route('/predict', methods=['POST'])
def predict():
    """Predict function"""
    # Get the raw inputs from the form
    raw_inputs = request.get_json()
    # Validate the raw inputs
    errors = validate_inputs(raw_inputs)
    if errors:
        return jsonify({'status': 'error', 'message': errors})
    # Prepare the input data for the model
    input_data = prepare_input_data(raw_inputs)
    # Make the prediction
    prediction = model.predict(input_data)
    # Get the predicted class
    predicted_index = list(model.classes_).index(prediction)
    # Get the predicted class name
    predicted_class = model.classes_[predicted_index]
    # Calculate the confidence
    confidence = float(model.predict_proba(input_data)[predicted_index][0]) * 100
    # Round the confidence to 2 decimal places
    confidence = round(confidence, 2)
    # Set the status and result text
    if predicted_class == 'APPROVED':
        status = 'Approved'
        result_text = 'Approved'
    else:
        status = 'Rejected'
        result_text = 'Rejected'
    # Build the explanation
    confidence_message = f'Confidence: {confidence}%'
    explanation_data = build_explanation(raw_inputs, status)
    # Prepare the applicant summary data
    applicant_data = {
        'Age': raw_inputs.get('Age'),
        'Income': int(raw_inputs.get('Income') or 0),
        'Married': int(raw_inputs.get('Married') or 0),
        'CreditScore': raw_inputs.get('CreditScore'),
        'DriverLicense': raw_inputs.get('DriverLicense'),
        'Vehicle': raw_inputs.get('Vehicle') or 'None'
    }
    # Log the prediction result
    logger.info('Prediction result: status=%s, confidence=%s', status, confidence)
    # Return the result
    return render_template(
        'result.html',
        prediction=predicted_class,
        status=status,
        confidence=confidence,
        confidence_message=confidence_message,
        explanation_data=explanation_data,
        applicant_data=applicant_data
    )

```

```

File Edit Selection View Go ... Smart bridge project
venv C:\Users\Ansh\Desktop\Python\Projects\SmartBridge\venv
app.py x introduction results x
# Importing Flask and Jinja2
import logging
import os
from flask import Flask, render_template, request

app = Flask(__name__)

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s %(levelname)s %(message)s'
)
logger = logging.getLogger(__name__)

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
MODEL_PATH = os.path.join(BASE_DIR, 'model.pkl')
ENCODER_PATH = os.path.join(BASE_DIR, 'encoder.pkl')

def load_artifacts():
    """Load the serialized model and encoders from local assets."""
    try:
        model = joblib.load(MODEL_PATH)
        encoders = joblib.load(ENCODER_PATH)
        logger.info('Loaded model from %s', MODEL_PATH)
        logger.info('Loaded encoders from %s', ENCODER_PATH)
        logger.info('Model classes: %s', list(getattr(model, 'classes_', [])))
        return model, encoders
    except Exception:
        logger.exception('Failed to load model or encoders')
        raise

model, encoders = load_artifacts()
feature_order = list(
    getattr(
        model,
        'feature_names_in_',
        [
            'Gender',
            'Age',
            'Married',
            'MarriedStatus',
            'Industry',
            'Ethnicity',
            'Transacted',
            'PriorDefault',
            'Delinquent',
            'CreditScore',
            'DriverLicense',
            'Vehicle'
        ]
    )
)

def parse_float(value):
    """Parse a numeric value to float, returning None for invalid input."""
    if value is None:
        return None
    try:
        parsed = float(str(value).strip())
    except (ValueError, TypeError):
        return None
    return parsed

def parse_int(value):
    """Parse a numeric value to int, returning None for invalid input."""
    if value is None:
        return None
    try:
        parsed = int(float(str(value).strip()))
    except (ValueError, TypeError):
        return None
    return parsed

def validate_inputs(raw_inputs):
    """Validate the raw values and return a list of user-facing errors."""
    errors = []
    # Check for required fields
    for field in categorical_fields:
        value = raw_inputs.get(field, '')
        if not value:
            errors.append(f'{field} is required.')
        elif field in categorical_fields:
            errors.append(f'Invalid value for {field}: {value}.')
    # Check for numeric fields
    age = parse_float(raw_inputs.get('Age', ''))
    if age is None:
        errors.append('Age must be a valid number.')
    elif age < 18 or age > 100:
        errors.append('Age must be between 18 and 100.')
    income = parse_float(raw_inputs.get('Income', ''))
    if income is None:
        errors.append('Annual Income must be a valid number.')
    elif income < 0:
        errors.append('Annual Income must be greater than zero.')
    debt = parse_float(raw_inputs.get('Debt', ''))
    if debt is None:
        errors.append('Debt must be a valid number.')
    return errors

```

```

File Edit Selection View Go ... Smart bridge project
venv C:\Users\Ansh\Desktop\Python\Projects\SmartBridge\venv
app.py x introduction results x
# Importing Flask and Jinja2
import logging
import os
from flask import Flask, render_template, request

app = Flask(__name__)

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s %(levelname)s %(message)s'
)
logger = logging.getLogger(__name__)

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
MODEL_PATH = os.path.join(BASE_DIR, 'model.pkl')
ENCODER_PATH = os.path.join(BASE_DIR, 'encoder.pkl')

def load_artifacts():
    """Load the serialized model and encoders from local assets."""
    try:
        model = joblib.load(MODEL_PATH)
        encoders = joblib.load(ENCODER_PATH)
        logger.info('Loaded model from %s', MODEL_PATH)
        logger.info('Loaded encoders from %s', ENCODER_PATH)
        logger.info('Model classes: %s', list(getattr(model, 'classes_', [])))
        return model, encoders
    except Exception:
        logger.exception('Failed to load model or encoders')
        raise

model, encoders = load_artifacts()
feature_order = list(
    getattr(
        model,
        'feature_names_in_',
        [
            'Gender',
            'Age',
            'Married',
            'MarriedStatus',
            'Industry',
            'Ethnicity',
            'Transacted',
            'PriorDefault',
            'Delinquent',
            'CreditScore',
            'DriverLicense',
            'Vehicle'
        ]
    )
)

def parse_float(value):
    """Parse a numeric value to float, returning None for invalid input."""
    if value is None:
        return None
    try:
        parsed = float(str(value).strip())
    except (ValueError, TypeError):
        return None
    return parsed

def parse_int(value):
    """Parse a numeric value to int, returning None for invalid input."""
    if value is None:
        return None
    try:
        parsed = int(float(str(value).strip()))
    except (ValueError, TypeError):
        return None
    return parsed

def validate_inputs(raw_inputs):
    """Validate the raw values and return a list of user-facing errors."""
    errors = []
    # Check for required fields
    for field in categorical_fields:
        value = raw_inputs.get(field, '')
        if not value:
            errors.append(f'{field} is required.')
        elif field in categorical_fields:
            errors.append(f'Invalid value for {field}: {value}.')
    # Check for numeric fields
    age = parse_float(raw_inputs.get('Age', ''))
    if age is None:
        errors.append('Age must be a valid number.')
    elif age < 18 or age > 100:
        errors.append('Age must be between 18 and 100.')
    income = parse_float(raw_inputs.get('Income', ''))
    if income is None:
        errors.append('Annual Income must be a valid number.')
    elif income < 0:
        errors.append('Annual Income must be greater than zero.')
    debt = parse_float(raw_inputs.get('Debt', ''))
    if debt is None:
        errors.append('Debt must be a valid number.')
    return errors

def predict():
    """Predict function"""
    # Get the raw inputs from the form
    raw_inputs = request.get_json()
    # Validate the raw inputs
    errors = validate_inputs(raw_inputs)
    if errors:
        return jsonify({'status': 'error', 'message': errors})
    # Prepare the input data for the model
    input_data = prepare_input_data(raw_inputs)
    # Make the prediction
    prediction = model.predict(input_data)
    # Get the predicted class
    predicted_index = list(model.classes_).index(prediction)
    # Get the predicted class name
    predicted_class = model.classes_[predicted_index]
    # Calculate the confidence
    confidence = float(model.predict_proba(input_data)[predicted_index][0]) * 100
    # Round the confidence to 2 decimal places
    confidence = round(confidence, 2)
    # Set the status and result text
    if predicted_class == 'APPROVED':
        status = 'Approved'
        result_text = 'Approved'
    else:
        status = 'Rejected'
        result_text = 'Rejected'
    # Build the explanation
    confidence_message = f'Confidence: {confidence}%'
    explanation_data = build_explanation(raw_inputs, status)
    # Prepare the applicant summary data
    applicant_data = {
        'Age': raw_inputs.get('Age'),
        'Income': int(raw_inputs.get('Income') or 0),
        'Married': int(raw_inputs.get('Married') or 0),
        'CreditScore': raw_inputs.get('CreditScore'),
        'DriverLicense': raw_inputs.get('DriverLicense'),
        'Vehicle': raw_inputs.get('Vehicle') or 'None'
    }
    # Log the prediction result
    logger.info('Prediction result: status=%s, confidence=%s', status, confidence)
    # Return the result
    return render_template(
        'result.html',
        prediction=predicted_class,
        status=status,
        confidence=confidence,
        confidence_message=confidence_message,
        explanation_data=explanation_data,
        applicant_data=applicant_data
    )

```

Conclusion and Future Scope

Conclusion

The Credit Card Approval Prediction using Machine Learning project successfully demonstrates how machine learning can be used to automate and improve the credit card approval process. By analyzing applicant information such as age, income, debt, employment status, credit score, marital status, and other financial attributes, the developed model accurately predicts whether a credit card application is likely to be Approved or Rejected. Multiple classification algorithms, including Logistic Regression, Decision Tree, and Random Forest, were evaluated, and Logistic Regression was selected as the final model for deployment. The model was integrated into a Flask-based web application with an interactive user interface that provides real-time predictions, confidence scores, and personalized explanations. This system helps reduce manual effort, improves consistency in decision-making, minimizes human bias, and enables faster credit approval assessments. Overall, the project demonstrates that machine learning is an effective solution for enhancing the efficiency, accuracy, and reliability of the credit card approval process.

Future Scope

The proposed Credit Card Approval Prediction system can be further enhanced with several advanced features to improve its performance and real-world usability. Future improvements include:

- Integrating the system with real-time banking and financial databases for live applicant verification
- Enhancing prediction accuracy by training on larger and more diverse datasets.
- Incorporating advanced machine learning and deep learning models for improved performance.
- Adding fraud detection mechanisms to identify suspicious or high-risk applications.
- Implementing user authentication and role-based access for secure access to the application.
- Deploying the application on cloud platforms with database support for better scalability and accessibility.
- Providing detailed decision explanations and personalized recommendations to help applicants improve their approval chances.